

10장 그래프 |





Q & A

- 언제라도 질문하세요
모르면 외우면 되지만 코딩 안되면 꼭 질문
 1. 대면수업
 2. 한성e-class 질의응답게시판

- 강사의 1번 선생님은 여러분의 질문





1. 그래프란?
2. 그래프의 정의와 용어
3. 그래프의 표현 방법
4. 그래프의 탐색
5. 깊이 우선 탐색
6. 너비 우선 탐색



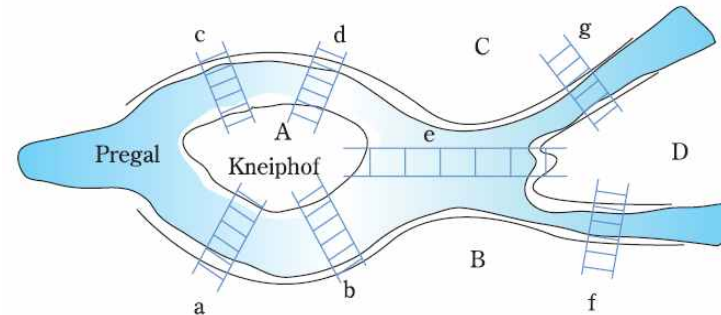
그래프 (graph)

- 연결되어 있는 객체 간의 관계를 표현하는 자료구조
 - ▣ (예) 우리가 배운 트리(**tree**)도 그래프의 특수한 경우임
 - ▣ (예) 지도에서 도시들의 연결 상태

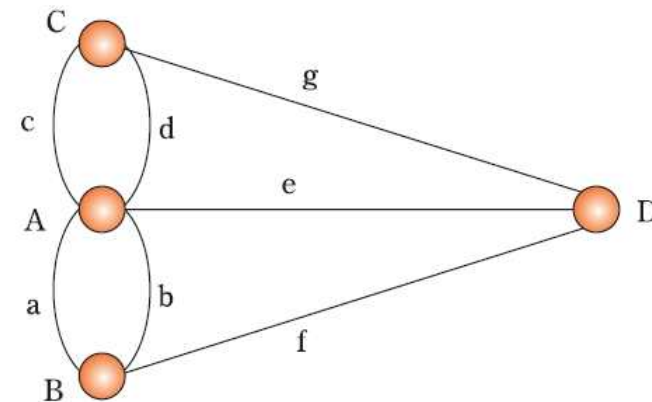


그래프 역사

- 1800년대 오일러에 의하여 창안
- 오일러 문제
 - ▣ 모든 다리를 **한번만** 건너서 처음 출발했던 장소로 **돌아오는** 문제
- A,B,C,D 지역의 연결 관계 표현
 - ▣ 위치: 정점(node)
 - ▣ 다리: 간선(edge)
- 오일러 정리
 - ▣ 모든 정점에 연결된 간선의 수가 짝수이면 오일러 경로 존재함 (증명함)
 - ▣ 따라서 그래프 (b)에는 오일러 경로가 존재하지 않음



(a) 모든 다리를 한 번만 건너 돌아오는 경로 문제



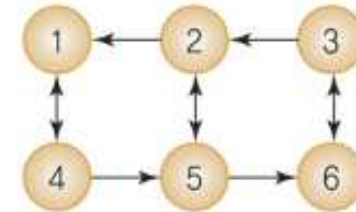
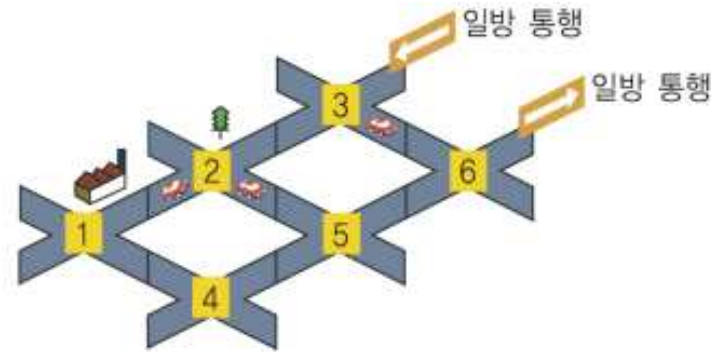
(b) 문제 (a)의 그래프 표현



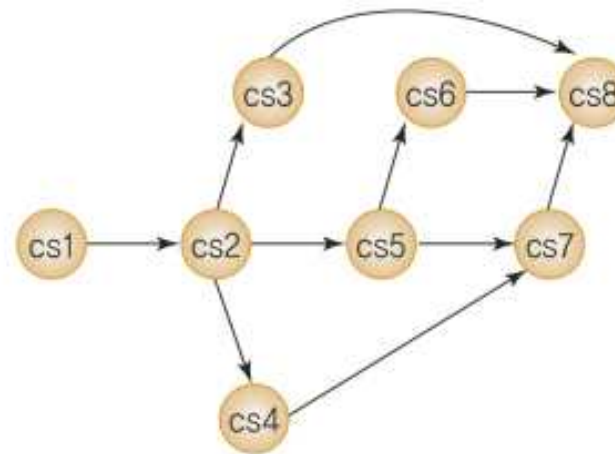


그래프로 표현하는 것들

□ 도로망

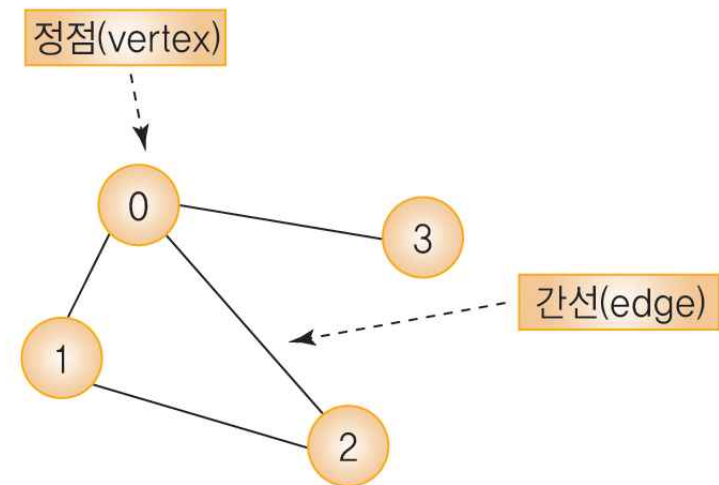


□ 선수과목 관계



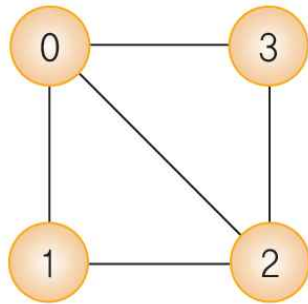
그래프 정의

- 그래프 G 는 (V, E) 로 표시
- 정점(vertices)
 - ▣ 여러 가지 특성을 가질 수 있는 객체 의미
 - ▣ $V(G)$: 그래프 G 의 정점들의 집합
 - ▣ 노드(node)라고도 불림
- 간선(edge)
 - ▣ 정점들 간의 관계 의미
 - ▣ $E(G)$: 그래프 G 의 간선들의 집합
 - ▣ 링크(link)라고도 불림

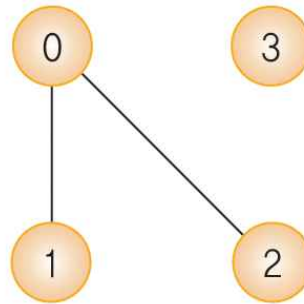




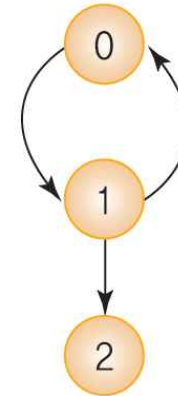
그래프 표현의 예



G1



G2



G3

$V(G1) = \{0, 1, 2, 3\}, \quad E(G1) = \{(0, 1), (0, 2), (0, 3), (1, 2), (2, 3)\}$

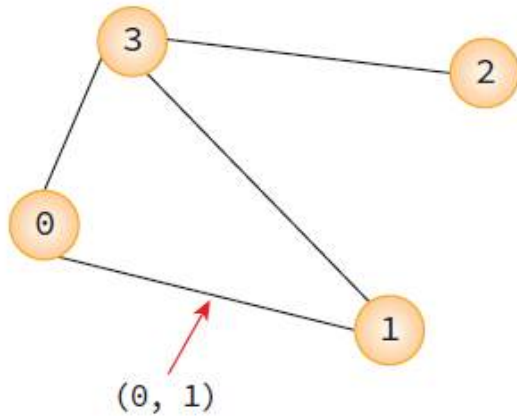
$V(G2) = \{0, 1, 2, 3\}, \quad E(G2) = \{(0, 1), (0, 2)\}$

$V(G3) = \{0, 1, 2\}, \quad E(G3) = \{<0, 1>, <1, 0>, <1, 2>\}$

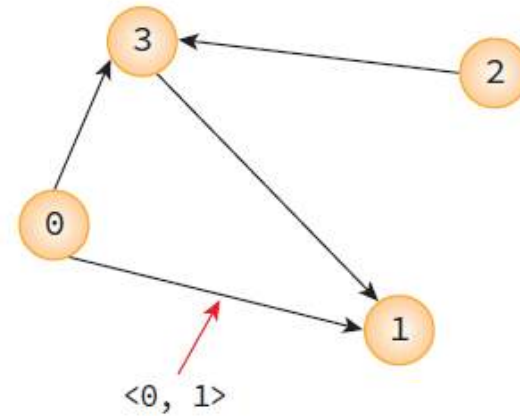




그래프의 종류



$V(G1) = \{0, 1, 2, 3\},$
 $E(G1) = \{(0, 1), (0, 3), (1, 3), (2, 3)\}$



$V(G2) = \{0, 1, 2, 3\},$
 $E(G2) = \{<0, 1>, <0, 3>, <3, 1>, <2, 3>\}$

[그림 10-9] 무방향 그래프와 방향 그래프

- 길 찾기 그래프에서
도보 길 찾기는 무방향 그래프
자동차 길 찾기는 방향 그래프 (일방통행 표시) 필요

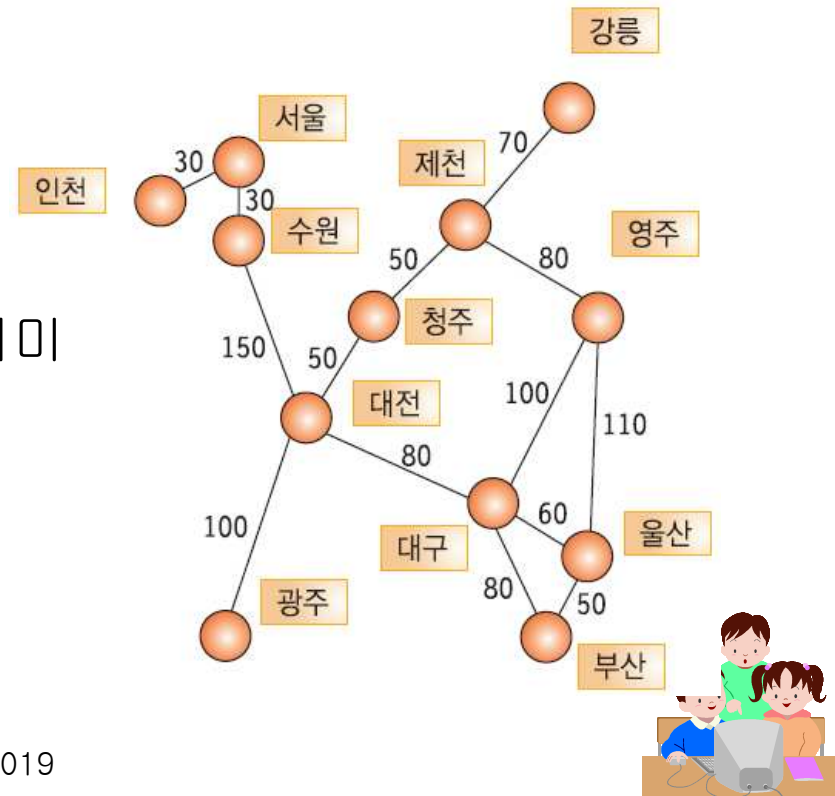




- 가중치 그래프(weighted graph)는 네트워크(network)라고도 함
- 간선에 비용(cost)이나 가중치(weight)가 할당된 그래프

- 네트워크 예

- ▣ 정점 : 각 도시를 의미
- ▣ 간선 : 도시를 연결하는 도로 의미
- ▣ 가중치 : 도로의 길이

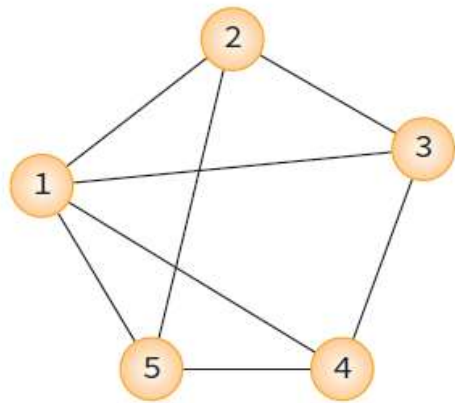




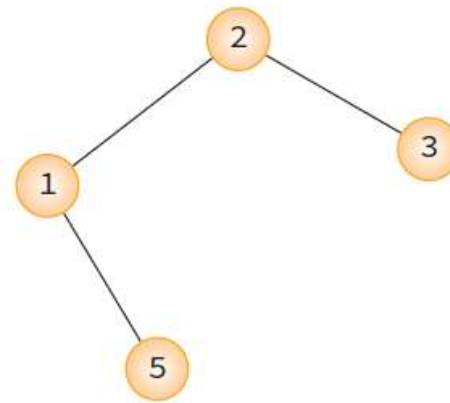
부분 그래프(subgraph)

- 정점 집합 $V(G)$ 와 간선 집합 $E(G)$ 의 부분 집합으로 이루어진 그래프
- 그래프 G_1 의 부분 그래프들

$$V(S) \subseteq V(G)$$
$$E(S) \subseteq E(G)$$



(a) 그래프

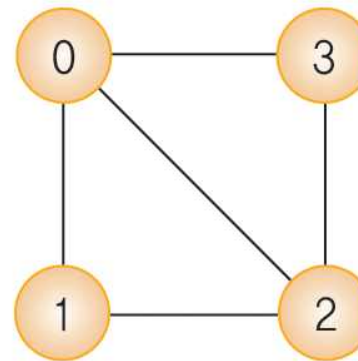


(b) 부분 그래프





- 인접 정점(adjacent vertex)
 - ▣ 하나의 정점에서 간선에 의해 직접 연결된 정점
 - ▣ G1에서 정점 0의 인접 정점: 정점 1, 정점 2, 정점 3
- 무방향 그래프의 차수(degree)
 - ▣ 하나의 정점에 연결된 다른 정점의 수
 - ▣ G1에서 정점 0의 차수: 3



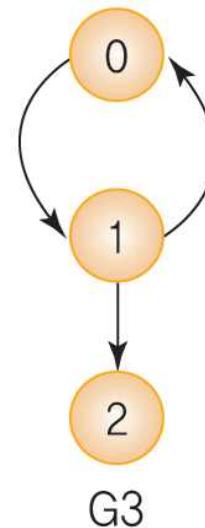
G1





□ 방향 그래프의 차수(degree)

- 진입 차수(in-degree) : 외부에서 오는 간선의 수
- 진출 차수(out-degree) : 외부로 향하는 간선의 수
- G3에서 정점 1의 차수: 내차수 1, 외차수 2
- 방향 그래프의 모든 진입(진출) 차수의 합은 간선의 수
 - G3의 진입 차수의 합: 3
 - G3의 진출 차수의 합: 3
 - G3의 간선 합: 3





그래프의 경로(path)

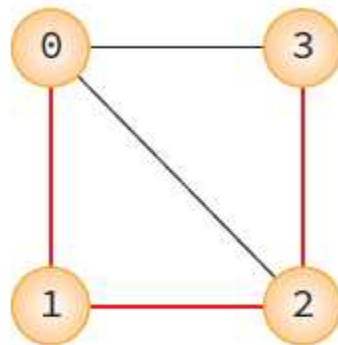
- 무방향 그래프의 정점 s 로부터 정점 e 까지의 경로
 - ▣ 정점의 나열 $s, v_1, v_2, \dots, v_k, e$
 - ▣ 나열된 정점들 간에 반드시 간선 $(s, v_1), (v_1, v_2), \dots, (v_k, e)$ 존재
- 단순 경로(simple path)
 - ▣ 경로 중에서 반복되는 간선이 없는 경로
- 사이클(cycle)
 - ▣ 단순 경로의 시작 정점과 종료 정점이 동일한 경로



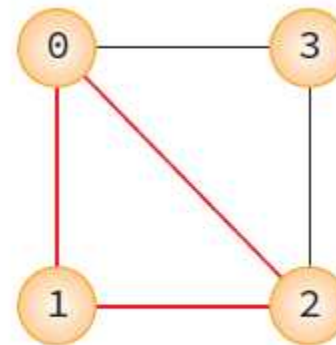


그래프의 경로(path)

- G1의 0, 1, 2, 3은 경로지만 0, 1, 3, 2는 경로 아님
- G1의 1, 0, 2, 3은 단순경로이지만 1, 0, 2, 0은 단순경로 아님
- G1의 0, 1, 2, 0는 사이클



단순경로: 0, 1, 2, 3



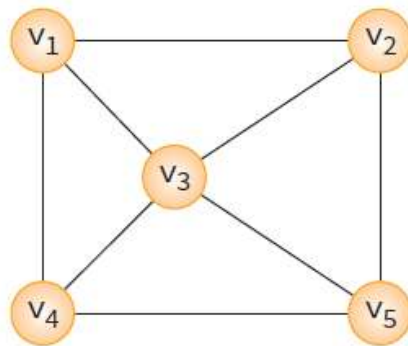
사이클: 0, 1, 2, 0



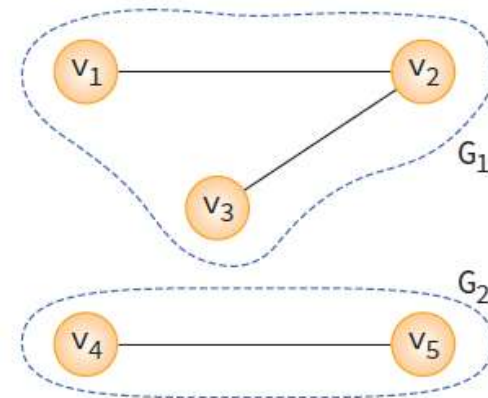


그래프의 연결정도

- 연결 그래프(connected graph)
 - ▣ 무방향 그래프 G 에 있는 모든 정점쌍에 대하여 항상 경로 존재
 - ▣ (a)는 연결 그래프, (b)는 비연결 그래프임



(a)



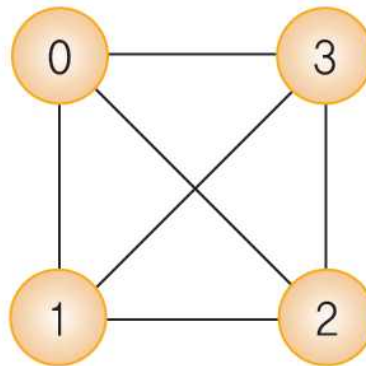
(b)





그래프의 연결정도

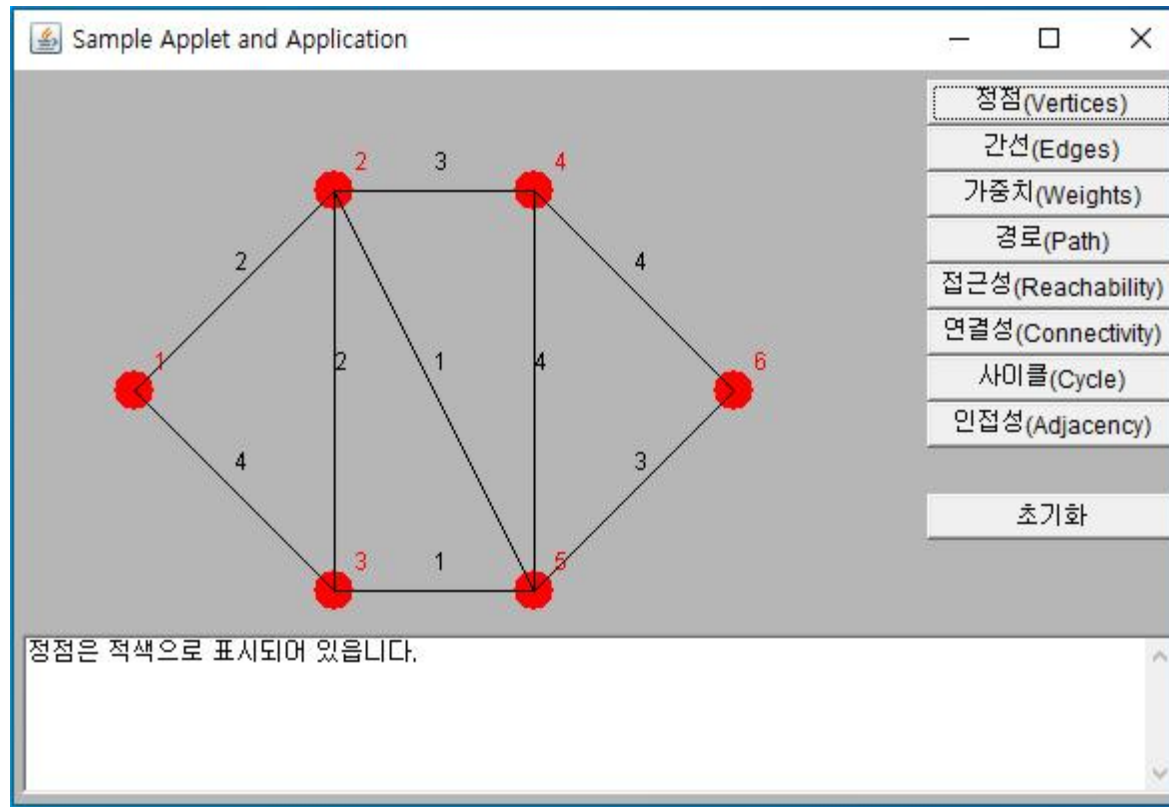
- 완전 그래프(complete graph)
 - ▣ 모든 정점이 직접 연결되어 있는 그래프
 - ▣ n개의 정점을 가진 무방향 완전그래프의 간선의 수: $n \times (n-1) / 2$
 - ▣ $n=4$, 간선의 수 = $(4 \times 3) / 2 = 6$





Graph 용어 가상실습

- Graph 용어를 28->GraphTerms로 알아봄





1. 그래프란?
2. 그래프의 정의와 용어
3. 그래프의 표현(구현) 방법
4. 그래프의 탐색
5. 깊이 우선 탐색
6. 너비 우선 탐색





.객체: 정점의 집합과 간선의 집합

.연산:

- `create_graph()` ::= 그래프를 생성한다.
- `init(g)` ::= 그래프 `g`를 초기화한다.
- `insert_vertex(g,v)` ::= 그래프 `g`에 정점 `v`를 삽입한다.
- `insert_edge(g,u,v)` ::= 그래프 `g`에 간선 `(u,v)`를 삽입한다.
- `delete_vertex(g,v)` ::= 그래프 `g`의 정점 `v`를 삭제한다.
- `delete_edge(g,u,v)` ::= 그래프 `g`의 간선 `(u,v)`를 삭제한다.
- `is_empty(g)` ::= 그래프 `g`가 공백 상태인지 확인한다.
- `adjacent(v)` ::= 정점 `v`에 인접한 정점들의 리스트를 반환한다.
- `destroy_graph(g)` ::= 그래프 `g`를 제거한다.





그래프 표현(구현) 방법

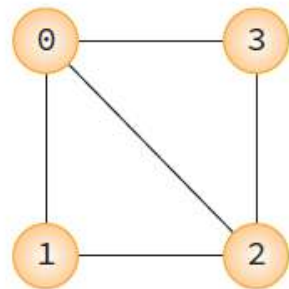
- 인접 행렬 (adjacent matrix) 방법
- 인접 리스트(adjacent list) 방법

스택자료구조, 큐, 리스트, 트리, 힙자료구조는 array 또는 linked list로 구현



□ 인접행렬 (adjacent matrix) 방법

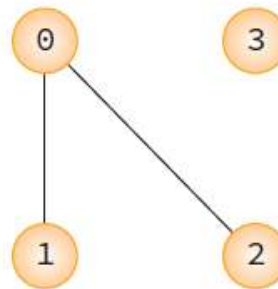
if(간선 (i, j)가 그래프에 존재) $M[i][j] = 1$,
그렇지 않으면 $M[i][j] = 0$.



정점 0의 차수
= 0+1+1+1=3

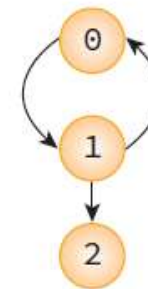
	0	1	2	3
0	0	1	1	1
1	1	0	1	0
2	1	1	0	1
3	1	0	1	0

(a)



	0	1	2	3
0	0	1	1	0
1	1	0	0	0
2	1	0	0	0
3	0	0	0	0

(b)



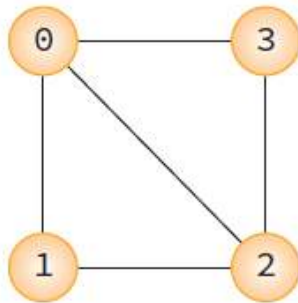
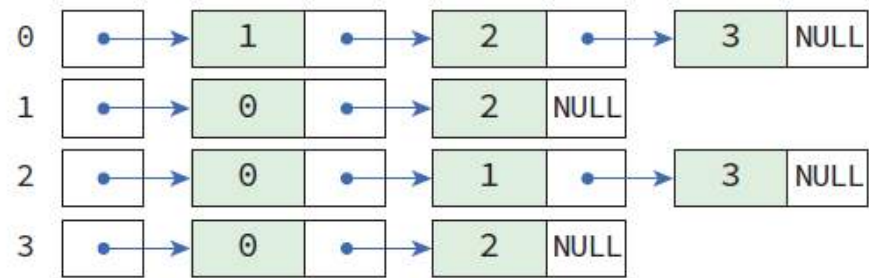
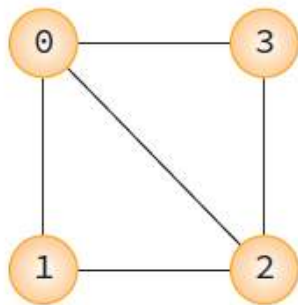
	0	1	2
0	0	1	0
1	1	0	1
2	0	0	0

(c)



인접 리스트 방법

- 인접리스트 (adjacency list) 방법
 - ▣ 각 정점에 인접한 정점들을 연결리스트로 표현





인접 행렬 구현 (adj_mat.c)

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 50
typedef struct GraphType {
    int n;          // 정점의 개수
    int adj_mat[MAX_VERTICES][MAX_VERTICES];
} GraphType;

// 그래프 초기화
void init(GraphType* g)
{
    int r, c;
    g->n = 0;
    for (r = 0; r < MAX_VERTICES; r++)
        for (c = 0; c < MAX_VERTICES; c++)
            g->adj_mat[r][c] = 0;
}
```





```
// 정점 삽입 연산
void insert_vertex(GraphType* g, int v)
{
    if (((g->n) + 1) > MAX_VERTICES) {
        fprintf(stderr, "그래프: 정점의 개수 초과");
        return;
    }
    g->n++;
}

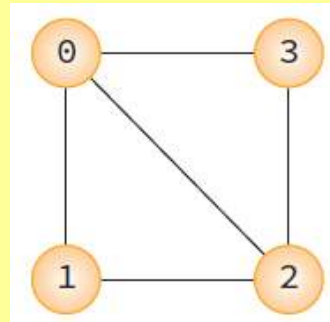
// 간선 삽입 연산
void insert_edge(GraphType* g, int start, int end)
{
    if (start >= g->n || end >= g->n) {
        fprintf(stderr, "그래프: 정점 번호 오류");
        return;
    }
    g->adj_mat[start][end] = 1;
    g->adj_mat[end][start] = 1;
}
```

이전 행렬 구현

```
void main()
{
    GraphType *g;
    g = (GraphType *)malloc(sizeof(GraphType));
    init(g);
    for(int i=0;i<4;i++)

        insert_vertex(g, i);
        insert_edge(g, 0, 1);
        insert_edge(g, 0, 2);
        insert_edge(g, 0, 3);
        insert_edge(g, 1, 2);
        insert_edge(g, 2, 3);
        print_adj_mat(g);

    free(g);
}
```



0	1	1	1
1	0	1	0
1	1	0	1
1	0	1	0





VS 솔루션 생성(프로젝트명: ch10 파일명 ch10.c)

원하는 작업을 선택하세요.

최근 파일 열기(R)

▲ 오늘



ch10.sln

2023-09-15

C:\Users\admin\source\repos\ch10





학생 > 교수님 ~ VS 변수창 보고 싶어요

1. 35번줄에 중단점 설정
2. 디버깅시작
3. start = 0 end = 1일때
4. g->adj_mat[0][1] 은 0
5. Step into 진행
6. g->adj_mat[0][1] 은 1

학생 > g->adj_mat[0][1] 가 생각한 대로 값이 변화하네요

g	0x00
n	4
adj_mat	0x00
[0]	0x00
[0]	0
[1]	0
[2]	0

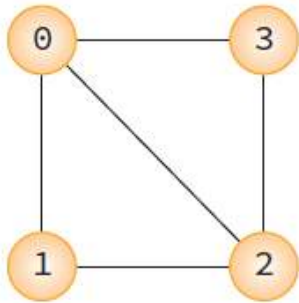
g	0x00
n	4
adj_mat	0x00
[0]	0x00
[0]	0
[1]	1
[2]	0

0	1	1	1
1	0	1	0
1	1	0	1
1	0	1	0





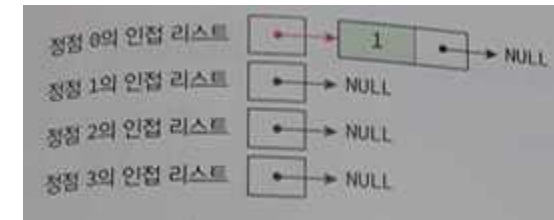
인접 리스트 구현 (adj_list.c)



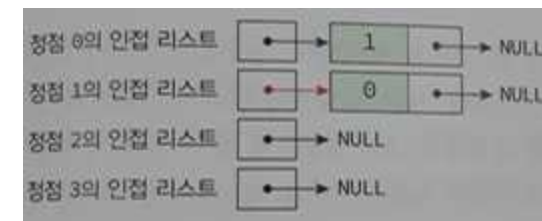
```
Microsoft Visual Studio 디버그 콘솔
정점 0의 인접 리스트 -> 3 -> 2 -> 1
정점 1의 인접 리스트 -> 2 -> 0
정점 2의 인접 리스트 -> 3 -> 1 -> 0
정점 3의 인접 리스트 -> 2 -> 0
```

Insert_edge(0,1) 호출

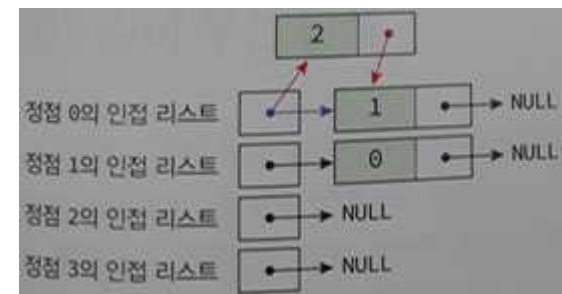
교재 377P



Insert_edge(1,0) 호출



Insert_edge(0,2) 호출





학생> 제가 인접리스트 자료구조 단계별 그림으로 만들었는데 VS에서
인접리스트 자료구조 보고 싶어요

1. 72번 줄에 중단점 설정
2. 디버깅 시작
3. Step into로 43번 줄까지 진행하여
vertex 2를 추가전 인접리스트 자료구조 확인
4. Step into로 47번 줄까지 진행하여
vertex 2를 추가후 인접리스트 자료구조 확인

학생> 교수님~ 제가 만든 인접리스트 자료구조
단계별 그림과 VS 인접리스트 자료구조를
비교해보니 일치합니다.

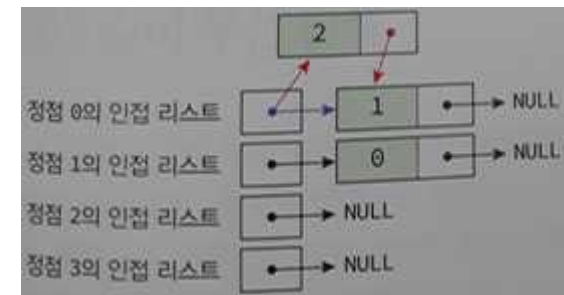
향후 에러가 발생하면 디버깅을 할 수 있습니다.

디버깅은 Line by Line 예측값과 실행값 비교인데

제가 만든 자료구조 그림으로 예측값 만들고

VS에서 한줄씩 실행하면서 실행값이 로컬창에 나오면
비교하여 에러 수정하겠습니다

Insert_edge(0,2) 호출



이름	값
g	0x01717c20 {n=4 adj_li
n	4
adj_list	0x01717c24 {0x017117
[0]	0x01711710 {vertex=1
vertex	1
link	0x00000000 <NULL>

이름	값
g	0x01717c20 {n=4 adj_li
n	4
adj_list	0x01717c24 {0x01711b
[0]	0x01711b58 {vertex=2
vertex	2
link	0x01711710 {vertex=1
vertex	1
link	0x00000000 <NULL>





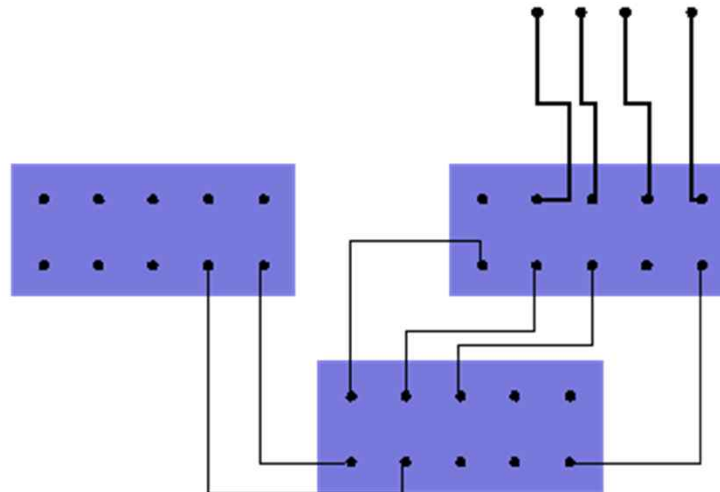
1. 그래프란?
2. 그래프의 정의와 용어
3. 그래프의 표현(구현) 방법
4. 그래프의 탐색
5. 깊이 우선 탐색
6. 너비 우선 탐색

* 정답이 있는 연습문제 홀수문제 꼭 해결해보세요



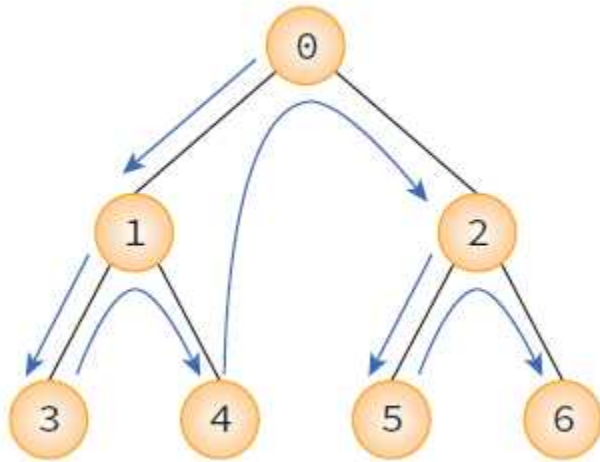
그래프 탐색

- 하나의 정점으로부터 시작하여 차례대로 **모든 정점들을 한번씩만** 방문 (무한루프에 빠지지 말아야 함: 미로탈출)
- 미로탈출문제에서 출구를 찾으려면 모든 길을 가보아야 하고 한번 간 길은 다시 방문하지 말아야 무한 루프에 빠지지 않음
- 많은 문제들이 단순히 그래프의 노드를 탐색하는 것으로 해결
(예) 도로망에서 특정 도시에서 다른 도시로 갈 수 있는지 여부
(예) 전자회로에서 특정 단자와 다른 단자가 서로 연결되어 있는지 여부

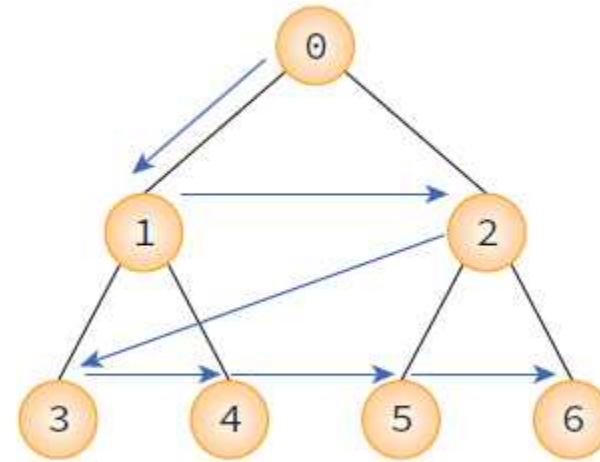


그래프 탐색

- 깊이 우선 탐색(DFS: depth first search)
- 너비 우선 탐색(BFS: breath first search)



깊이 우선 탐색



너비 우선 탐색

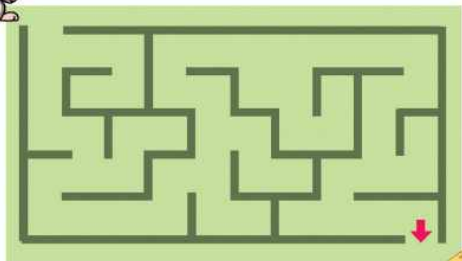




깊이 우선 탐색 (DFS)

□ 깊이 우선 탐색 (DFS: depth-first search)

- 한 방향으로 갈 수 있을 때까지 가다가 더 이상 갈 수 없게 되면 가장 가까운 갈림길로 돌아와서 이 곳으로부터 다른 방향으로 다시 탐색 진행
- 되돌아가기 위해서는 스택 필요(순환함수 호출로 묵시적인 스택 이용 가능)
- 미로탐출은 위에서 보면 출구가 보이지만 옆에서 보면 출구가 안보임. 나의 C 코드도 출구가 안보임. 하지만 나의 C코드는 반드시 출구를 찾아냄 ← 어떻게

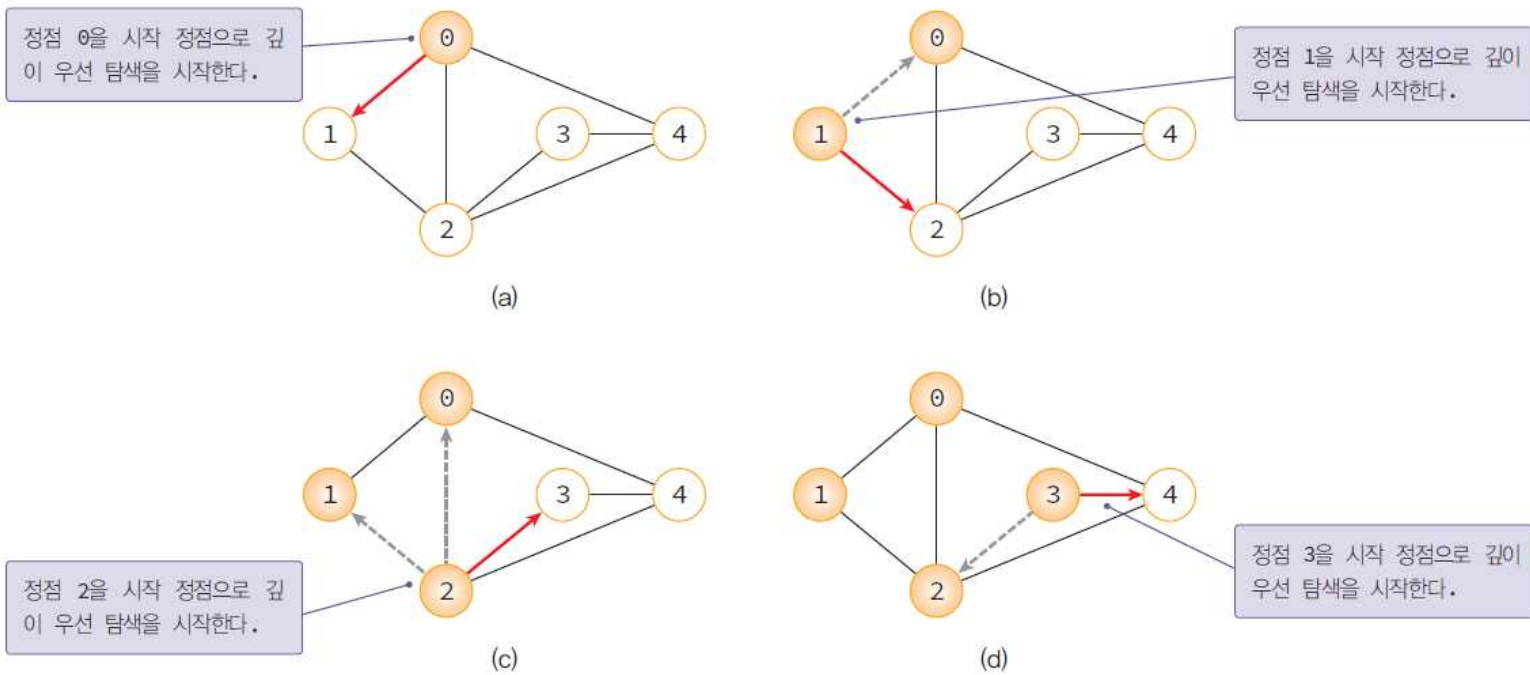


C

생능

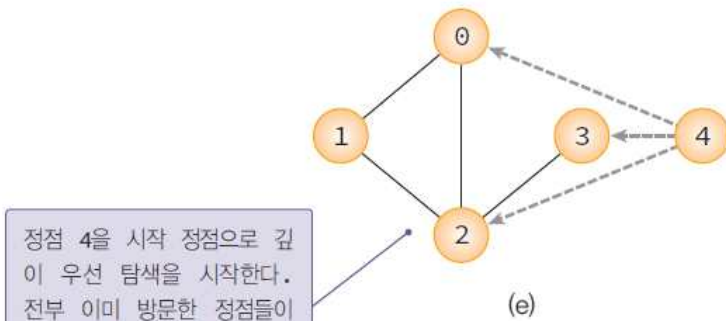


DFS 알고리즘

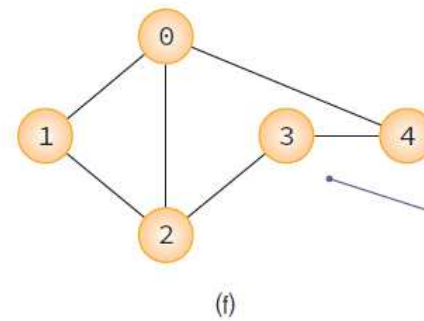




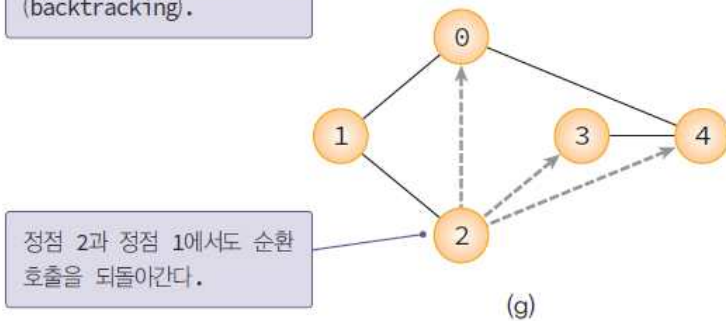
DFS 알고리즘



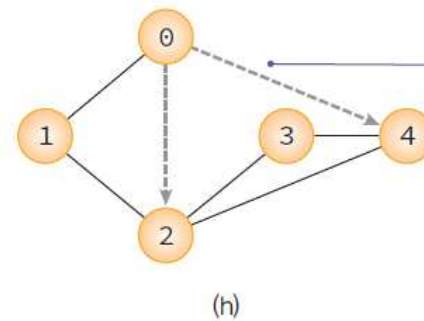
정점 4를 시작 정점으로 깊이 우선 탐색을 시작한다. 전부 이미 방문한 정점들이라 노드 3으로 되돌아간다 (backtracking).



정점 3에서도 더 이상 방문할 정점이 없어서 정점 순환 호출을 되돌아간다.



정점 2과 정점 1에서도 순환 호출을 되돌아간다.



정점 0에서 더 이상 방문할 정점이 없어서 탐색이 종료된다.





DFS 프로그램 (bfs.c)

```
// 인접 행렬로 표현된 그래프에 대한 깊이 우선 탐색
void dfs_mat(GraphType* g, int v)
{
    int w;
    visited[v] = TRUE;           // 정점 v의 방문 표시
    printf("정점 %d -> ", v);    // 방문한 정점 출력
    for (w = 0; w < g->n; w++)    // 인접 정점 탐색
        if (g->adj_mat[v][w] && !visited[w])
            dfs_mat(g, w);        // 정점 w에서 DFS 새로 시작
}
```



 DFS 프로그램

```
int main(void)
{
    GraphType *g;
    g = (GraphType *)malloc(sizeof(GraphType));
    init(g);
    for (int i = 0; i<4; i++)
        insert_vertex(g, i);
    insert_edge(g, 0, 1);
    insert_edge(g, 0, 2);
    insert_edge(g, 0, 3);
    insert_edge(g, 1, 2);
    insert_edge(g, 2, 3);

    printf("깊이 우선 탐색\n");
    dfs_mat(g, 0);
    printf("\n");
    free(g);
    return 0;
}
```





깊이 우선 탐색

정점 0 -> 정점 1 -> 정점 2 -> 정점 3 ->





DFS 가상실습

□ DFS가상실습을 27->BfsDfs로 알아봄

Sample Applet and Application

큐의 내용:

인접 행렬

0	1	1	0	1
1	0	1	0	0
1	1	0	1	1
0	0	1	0	1
1	0	1	1	0

v: 2 w: 0 adj_mat[v][w]: 1 visited[w]: 1

연산의 종류: DFS 수행속도: 10 정점의 개수: 5 가중치: 1

시작 중지 다음 초기화 모든 정점 삭제 인접행렬

Copyright (c) SCH Univ Image Lab 2005

```
void dfs_mat(int v)
{
    int w;
    visited[v] = 1;
    printf("%d ", v);
    for(w=0; w<n; w++)
        if( adj_mat[v][w] && !visited[w])
            dfs_mat(w);
}
```

연산의 종류 : DFS

정점의 개수 : 5

- 1) 마우스를 더블클릭하면 정점이 만들어진다.
- 2) 마우스로 정점들을 드래그하여 간선을 만든다
- 3) 정점을 이동하고자 하는 경우에는 **ctrl**키를 누른 채로 마우스로 원하는 정점을 드래그한다





dfs_change.c로 설명합니다.

학생> dfs.c를 왜 변경하시나요?

교수> 교재 그림10-19과 동일하게 하려합니다.

dfs.c는 정점4개 그래프인데 그림10-19은 정점5개 그래프
콘솔 결과와 그림 10-19와 일치

Microsoft Visual Studio 디버그 콘솔

= 인접행렬

	U0	U1	U2	U3	U4
U0	0	1	1	0	1
U1	1	0	1	0	0
U2	1	1	0	1	1
U3	0	0	1	0	1
U4	1	0	1	1	0

= 깊이 우선 탐색

정점 0 -> 정점 1 -> 정점 2 -> 정점 3 -> 정점 4 ->

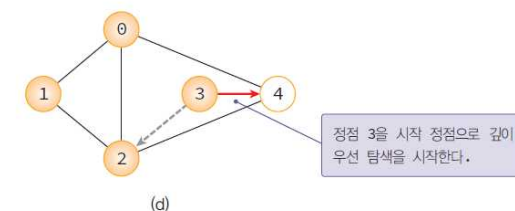
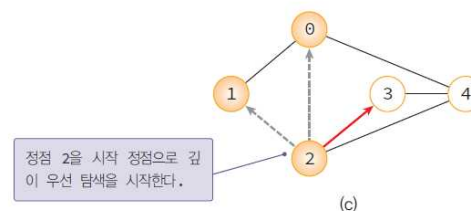
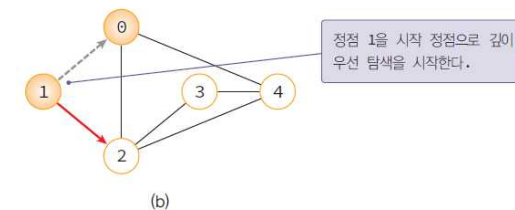
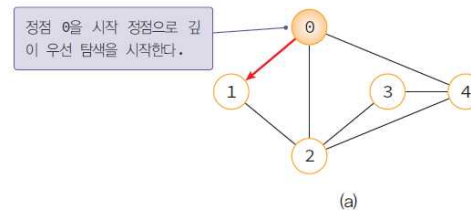
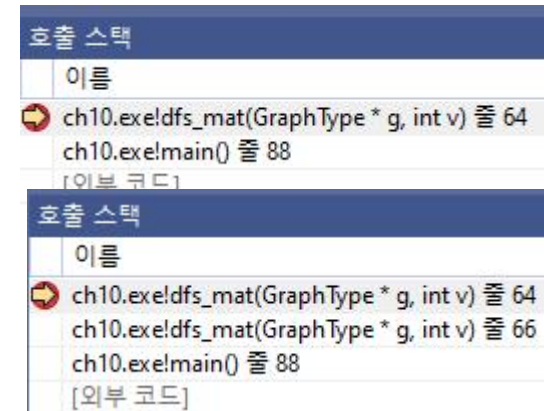




학생> 교수님~ 어려워요. 코드 한줄씩 설명해주세요.

교수> 코드만으로 한줄씩 설명이 어렵습니다. 자료구조 그림과 VS같이 (1/

1. dfs_change.c의 64번줄에 중단점 설정
2. 디버깅시작
3. 중단점 도달 후 로컬창에서 $v = 0$, v 값 printf출력. 그림(a) 상태
호출스택은 dfs_mat(0)임
4. 계속 버튼 로컬창에서 $v=1$, v 값 printf출력. 그림(b)상태
호출스택은 dfs_mat(1)임





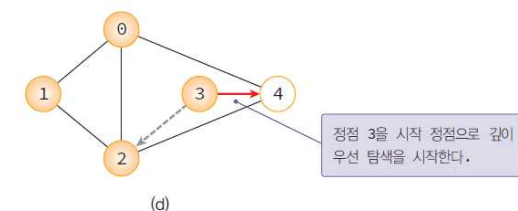
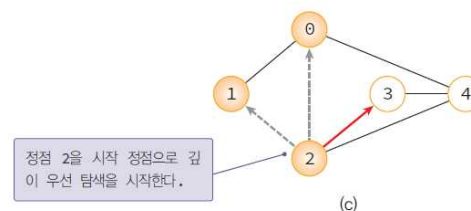
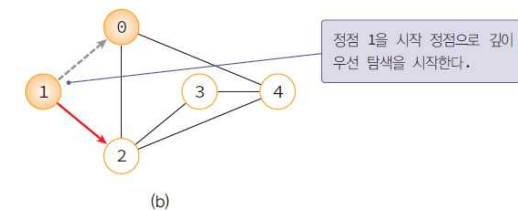
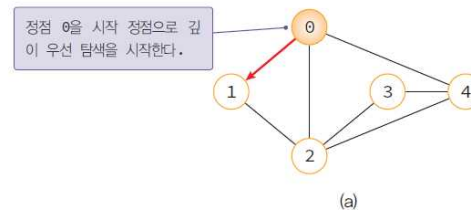
학생> 교수님~ 어려워요. 코드 한줄씩 설명해주세요.

교수> 코드만으로 한줄씩 설명이 어렵음. 자료구조 그림과 VS같이 (2/

1. 계속 버튼 로컬창에서 v=2, v값 printf출력. 그림(c)상태 호출스택은 dfs_mat(2)임
2. 계속 버튼 로컬창에서 v=3, v값 printf출력. 그림(d)상태 호출스택은 dfs_mat(3)임

호출 스택	
이름	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 48	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!main() 줄 68	

호출 스택	
이름	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 48	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!main() 줄 68	



C로 쉽게 풀어쓴 자료구조



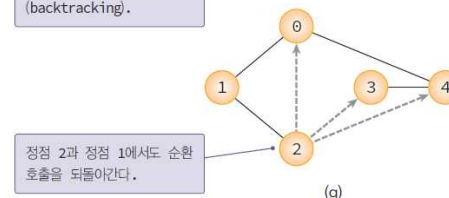
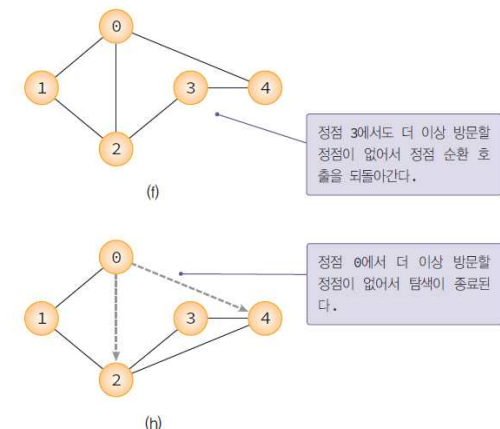
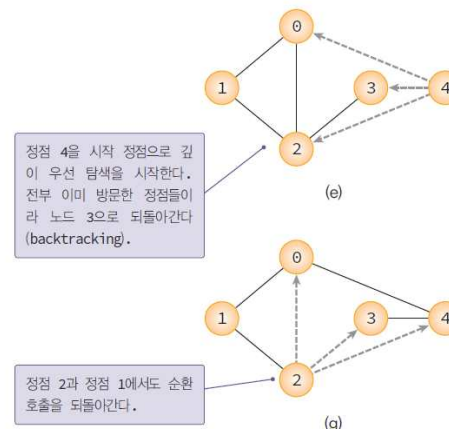
학생> 교수님~ 어려워요. 코드 한줄씩 설명해주세요.

교수> 코드만으로 한줄씩 설명이 어렵습니다. 자료구조 그림과 VS같이 (3/

1. 계속 버튼 로컬창에서 $v=4$, v 값 printf출력. 그림(e)상태 호출스택은 `dfs_mat(4)`임
2. Step Into로 진행하면 `dfs_mat()` 호출이 발생하지 않고 `return`이 되어 로컬창에서 $v=3$. 그림(f)상태 호출스택은 `dfs_mat()`가 5개에서 4개로 줄어듦

호출 스택	
이름	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 48	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!main() 줄 68	

호출 스택	
이름	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!main() 줄 68	





학생> 교수님~ 어려워요. 코드 한줄씩 설명해주세요.

교수> 코드만으로 한줄씩 설명이 어렵음. 자료구조 그림과 VS같이(4/

1. Step Into로 진행하면 dfs_mat() 호출이 발생하지 않고 return이 되어 로컬창에서 v=2. 그림(g)상태

호출스택은 dfs_mat()가 4개에서 3개로 줄어듬

2. Step Into로 진행하면 dfs_mat() 호출이 발생하지 않고 return이 되어 로컬창에서 v=1. 그림(g)와 (h) 사이 상태

호출스택은 dfs_mat()가 3개에서 2개로 줄어듬

3. Step Into로 진행하면 dfs_mat() 호출이 발생하지 않고 return이 되어 로컬창에서 v=0. 그림(h) 사이 상태

호출스택은 dfs_mat()가 2개에서 1개로 줄어듬

4. Step Into로 진행하면 dfs_mat() 호출이 발생하지 않고 return이 되어 main()으로 되돌아감

호출스택은 dfs_mat()가 1개에서 0개로

호출 스택

이름	주소
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!main() 줄 68	

호출 스택

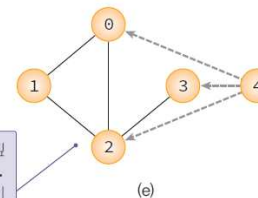
이름	주소
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!main() 줄 68	

호출 스택

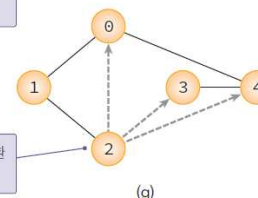
이름	주소
ch10.exe!dfs_mat(GraphType * g, int v) 줄 50	
ch10.exe!main() 줄 68	

호출 스택

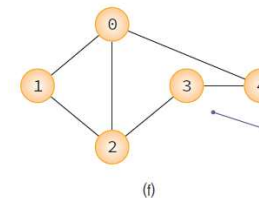
이름	주소
ch10.exe!main() 줄 68	



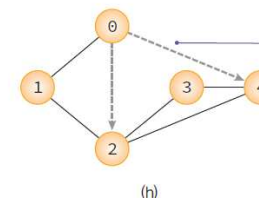
정점 4를 시작 정점으로 깊이 우선 탐색을 시작한다. 전부 이미 방문한 정점들이라 노드 3으로 되돌아간다 (backtracking).



정점 2과 정점 1에서도 순환 호출을 되돌아간다.



정점 3에서도 더 이상 방문할 정점이 없어서 정점 순환 호출을 되돌아간다.



정점 0에서 더 이상 방문할 정점이 없어서 탐색이 종료된다.



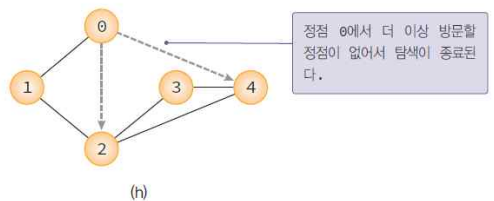
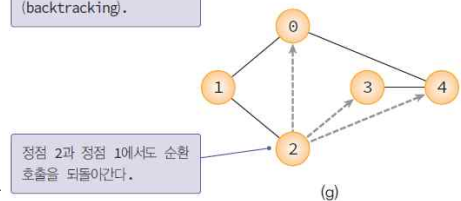
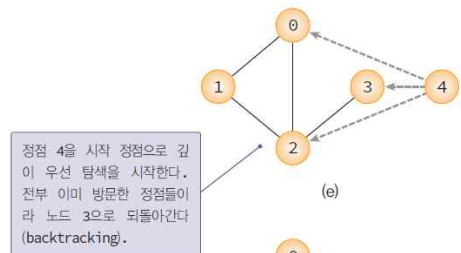
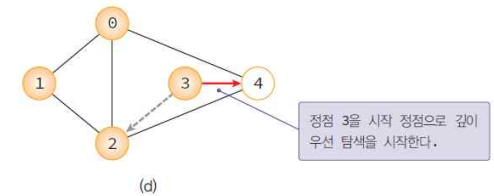
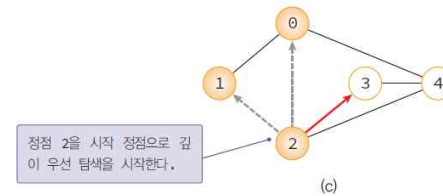
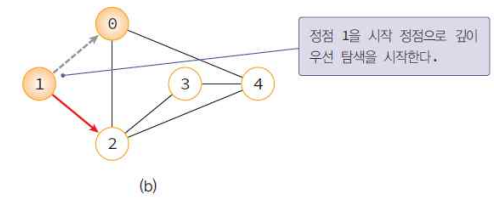
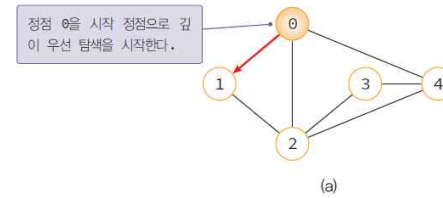
DFS 코드 설명 정리

1. 그림(a)~(d)는 깊이 우선 탐색과정으로 66번줄 dfs_mat()를 호출하고 그림(e)~(h)는 호출에서 retrun 하면서 방문하지 않은 인접정점을 64번줄에서점검한다

2. 하나의 정점으로부터 시작하여 깊이 우선으로 모든 정점(64번줄)들을 한번씩(65번줄)만 방문한다

g->n 은 vertex 갯수

```
58 // 인접 행렬로 표현된 그래프에 대한 깊이 우선 탐색
59 void dfs_mat(GraphType* g, int v)
60 {
61     int w;
62     visited[v] = TRUE; // 정점 v의 방문 표시
63     printf("정점 %d -> ", v); // 방문한 정점 출력
64     for (w = 0; w < g->n; w++) // 인접 정점 탐색
65         if (g->adj_mat[v][w] && !visited[w])
66             dfs_mat(g, w); //정점 w에서 DFS 새로 시작
67 }
68
```





DFS 프로그램(인접 리스트 버전)(bfs_list.c)

```
int visited[MAX_VERTICES];

// 인접 리스트로 표현된 그래프에 대한 깊이 우선 탐색
void dfs_list(GraphType* g, int v)
{
    GraphNode* w;
    visited[v] = TRUE;           // 정점 v의 방문 표시
    printf("정점 %d -> ", v);   // 방문한 정점 출력
    for (w = g->adj_list[v]; w; w = w->link) // 인접 정점 탐색
        if (!visited[w->vertex])
            dfs_list(g, w->vertex); // 정점 w에서 DFS 새로 시작
}
```





학생> 교수님~ dfs_list.c에 main()이 없어요

교수> dfs_list_change.c를 만들었어요

학생> 어떻게요

교수> adj_list.c에 dfs_list.c를 합쳤습니다

인접배열방식의 DFS와 동일한 결과가 나옵니다

dfs_change.c와 dfs_list_change.c 코드를 비교하면

정점 v의 방문 표시 단계, 인접 정점 탐색단계등이 동일하게 있는 것을 알 수 있습니다.

```
Microsoft Visual Studio 디버그 콘솔

= 인접리스트
정점 0의 인접 리스트 -> 1 -> 2 -> 4 -> NULL
정점 1의 인접 리스트 -> 2 -> 0 -> NULL
정점 2의 인접 리스트 -> 3 -> 4 -> 1 -> 0 -> NULL
정점 3의 인접 리스트 -> 4 -> 2 -> NULL
정점 4의 인접 리스트 -> 3 -> 2 -> 0 -> NULL

= 깊이 우선 탐색
정점 0 -> 정점 1 -> 정점 2 -> 정점 3 -> 정점 4 ->
```

```
58 // 인접 행렬로 표현된 그래프에 대한 깊이 우선 탐색
59 void dfs_mat(GraphType* g, int v)
60 {
61     int w;
62     visited[v] = TRUE; // 정점 v의 방문 표시
63     printf("정점 %d -> ", v); // 방문한 정점 출력
64     for (w = 0; w < g->n; w++) // 인접 정점 탐색
65         if (g->adj_mat[v][w] && !visited[w])
66             dfs_mat(g, w); //정점 w에서 DFS 새로 시작
67 }
```

```
67 // 인접 리스트로 표현된 그래프에 대한 깊이 우선 탐색
68 void dfs_list(GraphType* g, int v)
69 {
70     GraphNode* w;
71     visited[v] = TRUE; // 정점 v의 방문 표시
72     printf("정점 %d -> ", v); // 방문한 정점 출력
73     for (w = g->adj_list[v]; w; w = w->link) // 인접 정점 탐색
74         if (!visited[w->vertex])
75             dfs_list(g, w->vertex); //정점 w에서 DFS 새로 시작
76 }
```





깊이 우선 탐색의 시간 복잡도 분석

- 인접 행렬 $\rightarrow O(n^2)$
64번줄에서 n 번의 loop를 수행하므로 $O(n)$. n 개의 정점을 모두 방문해야 하므로 $n \cdot O(n) \rightarrow O(n^2)$
- 인접 리스트 $\rightarrow O(n+e)$
73번줄에서 각 정점에 연결되어 있는 간선의 개수만큼 탐색을 하게 되므로 시간복잡도 계산이 불가능 하다. 하지만 DFS가 다 끝난 후를 생각하면, 모든 정점을 한번씩 다 방문하고, 모든 간선을 한번씩 모두 검사했다고 할 수 있으므로 $O(n+e)$ 의 시간이 걸렸다고 할 수 있다.

```
58 // 인접 행렬로 표현된 그래프에 대한 깊이 우선 탐색
59 void dfs_mat(GraphType* g, int v)
60 {
61     int w;
62     visited[v] = TRUE; // 정점 v의 방문 표시
63     printf("정점 %d -> ", v); // 방문한 정점 출력
64     for (w = 0; w < g->n; w++) // 인접 정점 탐색
65         if (g->adj_mat[v][w] && !visited[w])
66             dfs_mat(g, w); //정점 w에서 DFS 새로 시작
67 }
```

```
67 // 인접 리스트로 표현된 그래프에 대한 깊이 우선 탐색
68 void dfs_list(GraphType* g, int v)
69 {
70     GraphNode* w;
71     visited[v] = TRUE; // 정점 v의 방문 표시
72     printf("정점 %d -> ", v); // 방문한 정점 출력
73     for (w = g->adj_list[v]; w; w = w->link) // 인접 정점 탐색
74         if (!visited[w->vertex])
75             dfs_list(g, w->vertex); //정점 w에서 DFS 새로 시작
76 }
```



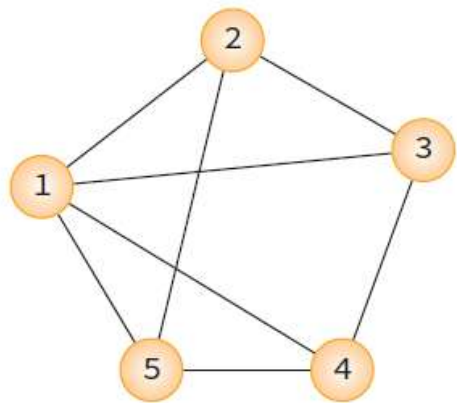


학생 > 표이우선탐색에서 인접행렬, 인접리스트 어느게 빠르나요?

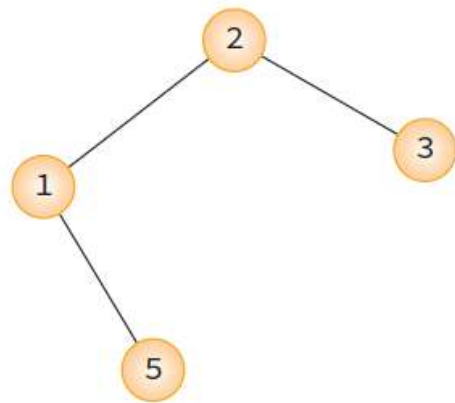
- 인접 행렬 $\rightarrow O(n^2)$
- 인접 리스트 $\rightarrow O(n+e)$

인접리스트는 **edge**가 많을 수록 시간소요되지만 인접행렬은 **node**의 제공으로 시간소요되기에 대부분의 그래프에서 인접리스트가 빠름

(a)그래프에서 인접행렬 $O(5^2) \Rightarrow O(25)$ 인접리스트 $O(5+8) \Rightarrow O(13)$



(a) 그래프



(b) 부분 그래프





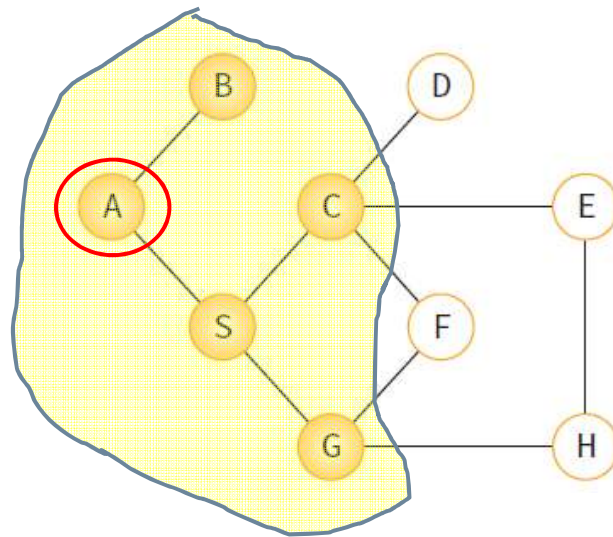
1. 그래프란?
2. 그래프의 정의와 용어
3. 그래프의 표현 방법
4. 그래프의 탐색
5. 깊이 우선 탐색
6. 너비 우선 탐색





너비우선 탐색(BFS)

- 너비 우선 탐색(BFS: breadth-first search)
 - ▣ 시작 정점으로부터 가까운 정점을 먼저 방문하고 멀리 떨어져 있는 정점을 나중에 방문하는 순회 방법
 - ▣ 큐를 사용하여 구현됨 (DFS는 스택으로 구현됨)



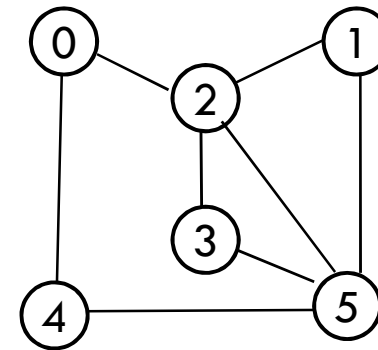
BFS는 시작 정점 A에서 가까운 정점들을 먼저 방문하는 기법이다.





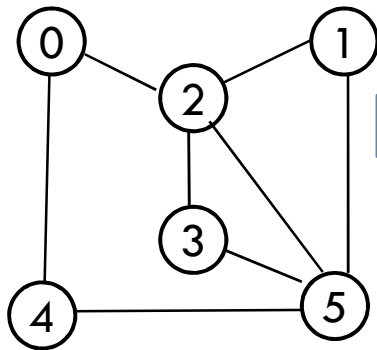
너비우선 탐색(BFS)

- 1단계: 시작점 0부터 거리가 0인 시작점 0 방문
2단계: 시작점 0부터 거리가 1인 정점 2, 4 방문
3단계: 시작점 0부터 거리가 2인 정점 1, 3, 5 방문



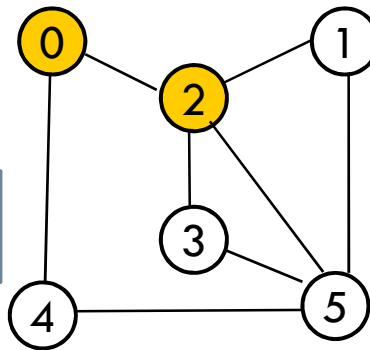


너비우선 탐색(BFS)



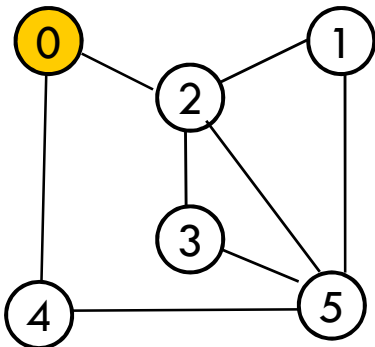
0					
---	--	--	--	--	--

처음에는 시작점인 0을 방문처리하고 큐에 추가한다



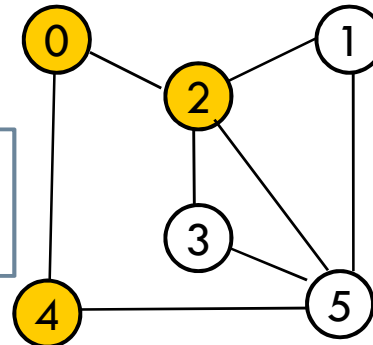
4	1	3	5		
---	---	---	---	--	--

큐에서 2를 꺼내서 인접점들을 방문처리하고 큐에 추가한다 인접점인 0은 방문했으므로 추가하지 않는다



2	4				
---	---	--	--	--	--

큐에서 0을 꺼내서 인접점들을 방문처리하고 큐에 추가한다



1	3	5			
---	---	---	--	--	--

큐에서 4를 꺼내서 인접점들을 방문처리하고 큐에 추가한다 인접점 0과 5중에 0과 5는 이미 방문처리했으므로 추가하지 않는다





BFS 프로그램 (인접행렬) (bfs.c)

```
void bfs_mat(GraphType* g, int v)
{
    int w;
    QueueType q;

    queue_init(&q);    // 큐 초기화
    visited[v] = TRUE;    // 정점 v 방문 표시
    printf("%d 방문 -> ", v);
    enqueue(&q, v);    // 시작 정점을 큐에 저장
    while (!is_empty(&q)) {
        v = dequeue(&q);    // 큐에 정점 추출
        for (w = 0; w < g->n; w++)    // 인접 정점 탐색
            if (g->adj_mat[v][w] && !visited[w]) {
                visited[w] = TRUE;    // 방문 표시
                printf("%d 방문 -> ", w);
                enqueue(&q, w);    // 방문한 정점을 큐에 저장
            }
    }
}
```





BFS 프로그램 (이접행렬)

```
int main(void)
{
    GraphType *g;
    g = (GraphType *)malloc(sizeof(GraphType));
    graph_init(g);
    for (int i = 0; i<6; i++)
        insert_vertex(g, i);
    insert_edge(g, 0, 2);
    insert_edge(g, 2, 1);
    insert_edge(g, 2, 3);
    insert_edge(g, 0, 4);
    insert_edge(g, 4, 5);
    insert_edge(g, 1, 5);

    printf("너비 우선 탐색\n");
    bfs_mat(g, 0);
    printf("\n");
    free(g);
    return 0;
}
```





너비 우선 탐색

0 방문 -> 2 방문 -> 4 방문 -> 1 방문 -> 3 방문 -> 5 방문 ->





BFS 가상실습

□ BFS가상실습을 27->BfsDfs로 알아봄

Sample Applet and Application

큐의 내용:

인접 행렬

0	1	1	0	1
1	0	1	0	0
1	1	0	1	1
0	0	1	0	1
1	0	1	1	0

v: 2 w: 0 adj_mat[v][w]: 1 visited[w]: 1

연산의 종류: DFS 수행속도: 10 정점의 개수: 5 가중치: 1

시작 중지 다음 초기화 모든 정점 삭제 인접행렬

Copyright (c) SCH Univ Image Lab 2005

```
void dfs_mat(int v)
{
    int w;
    visited[v] = 1;
    printf("%d ", v);
    for(w=0; w<n; w++)
        if( adj_mat[v][w] && !visited[w])
            dfs_mat(w);
}
```

연산의 종류 : BFS

정점의 개수 : 5

- 1) 마우스를 더블클릭하면 정점이 만들어진다.
- 2) 마우스로 정점들을 드래그하여 간선을 만든다
- 3) 정점을 이동하고자 하는 경우에는 **ctrl**키를 누른 채로 마우스로 원하는 정점을 드래그한다





bfs_change.c로 설명합니다.

학생> bfs.c를 왜 변경하시나요?

교수> 코드 이해를 높이기 위해 인접행렬도 출력했습니다.

```
Microsoft Visual Studio 디버그 콘솔
= 인접행렬
  U0 U1 U2 U3 U4 U5
U0  0  0  1  0  1  0
U1  0  0  1  0  0  1
U2  1  1  0  1  0  1
U3  0  0  1  0  0  1
U4  1  0  0  0  0  1
U5  0  1  1  1  1  0

너비 우선 탐색
0 방문 -> 2 방문 -> 4 방문 -> 1 방문 -> 3 방문 -> 5 방문 ->
```



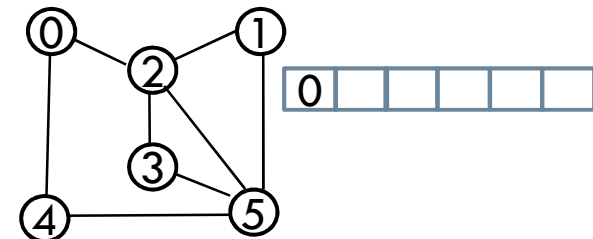
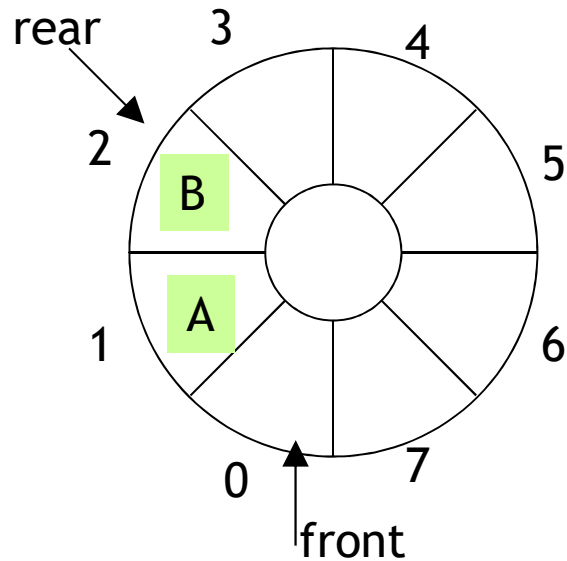


학생> 교수님~ 어려워요. 코드 한줄씩 설명해주세요.

교수> 코드만으로 한줄씩 설명이 어려움. 자료구조 그림과 VS같이(1/

1. bfs_change.c의 118번줄에 중단점 설정
2. 디버깅시작
3. 중단점 도달 후 로컬창에서 front=0, rear =1, queue[1] = 0, 방문노드 printf출력 그림(a) 상태

로컬	
검색(Ctrl+E)	
이름	값
g	0x011fed18 {
q	{queue=0x00
queue	0x00defa2c {
[0]	-858993460
[1]	0
[2]	-858993460



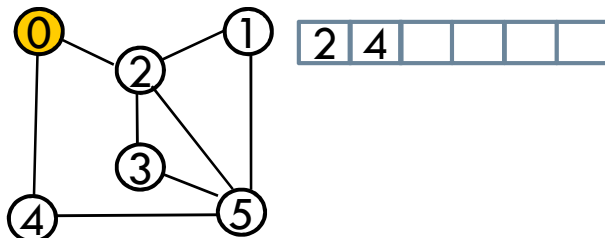


학생> 교수님~ 어려워요. 코드 한줄씩 설명해주세요.

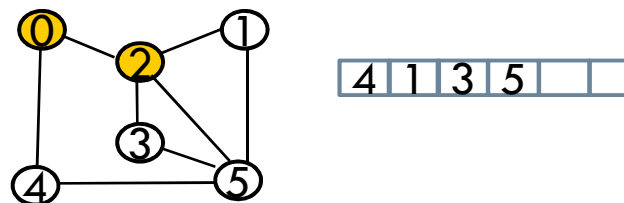
교수> 코드만으로 한줄씩 설명이 어려움. 자료구조 그림과 VS같이(2/

1. 계속버튼 로컬창에서 front=1, rear=3, queue[2] =2, queue[3]=4, 방문노드 printf출력 그림(b)상태
2. 계속버튼 로컬창에서 front=2, rear=6, queue[3] =4, queue[4]=1, queue[5]=3, queue[6]=5, 방문노드 printf출력 그림(c) 상태
2. 계속버튼 로컬창에서 front=3, rear=6, queue[4]=1, queue[5]=3, queue[6]=5, 방문노드 printf출력 그림(d)상태

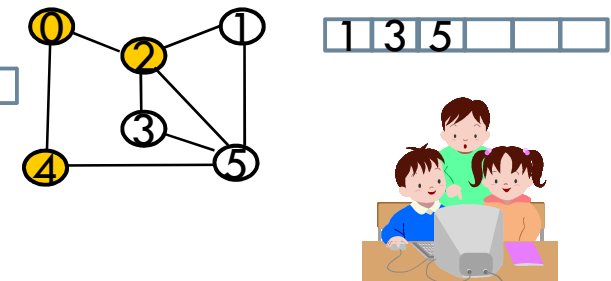
queue	0x0
[0]	-851
[1]	0
[2]	2
[3]	4
[3]	4
[4]	1
[5]	3
[6]	5
[7]	-858
[3]	4
[4]	1
[5]	3
[6]	5
[7]	-85



C로 쉽게 풀어쓴 자료구조



© 생능출판사 2019





BFS 프로그램(인접리스트) (bfs_list.c)

```
void bfs_list(GraphType* g, int v)
{
    GraphNode* w;
    QueueType q;

    init(&q); // 큐 초기화
    visited[v] = TRUE; // 정점 v 방문 표시
    printf("%d 방문 -> ", v);
    enqueue(&q, v); // 시작정점을 큐에 저장
    while (!is_empty(&q)) {
        v = dequeue(&q); // 큐에 저장된 정점 선택
        for (w = g->adj_list[v]; w; w = w->link) //인접 정점 탐색
            if (!visited[w->vertex]) { // 미방문 정점 탐색
                visited[w->vertex] = TRUE; // 방문 표시
                printf("%d 방문 -> ", w->vextex);
                enqueue(&q, w->vertex); //정점을 큐에 삽입
            }
    }
}
```





학생> 교수님~ bfs_list.c에 main()이 없어요

교수> bfs_list_change.c를 만들었어요

학생> 어떻게요

교수> adj_list.c와 bfs_list.c 그리고 bfs_change.c의 queue관련 함수를 합쳤습니다

인접배열방식의 BFS와 동일한 결과가 나옵니다

bfs_change.c와 bfs_list_change.c 코드를 비교하면

정점 v의 방문 표시 단계, 인접 정점 탐색단계등이 동일하게 있는 것을 알 수 있습니다.

Microsoft Visual Studio 디버그 콘솔

```
= 인접리스트
정점 0의 인접 리스트 -> 1 -> 2 -> 4 -> NULL
정점 1의 인접 리스트 -> 2 -> 0 -> NULL
정점 2의 인접 리스트 -> 3 -> 4 -> 1 -> 0 -> NULL
정점 3의 인접 리스트 -> 4 -> 2 -> NULL
정점 4의 인접 리스트 -> 3 -> 2 -> 0 -> NULL

= 깊이 우선 탐색
정점 0 -> 정점 1 -> 정점 2 -> 정점 3 -> 정점 4 ->
```

```
108 void bfs_mat(GraphType* g, int v)
109 {
110     int w;
111     QueueType q;
112
113     queue_init(&q); // 큐 초기화
114     visited[v] = TRUE; // 정점 v 방문 표시
115     printf("%d 방문 -> ", v);
116     enqueue(&q, v); // 시작 정점을 큐에 저장
117     while (!is_empty(&q)) {
118         v = dequeue(&q); // 큐에 정점 추출
119         for (w = 0; w < g->n; w++) // 인접 정점
120             if (g->adj_mat[v][w] && !visited[w])
121                 visited[w] = TRUE; // 방문 표시
122                 printf("%d 방문 -> ", w);
123                 enqueue(&q, w); // 방문한 정점
124     }
125 }
126
```

```
118 void bfs_list(GraphType* g, int v)
119 {
120     GraphNode* w;
121     QueueType q;
122
123     queue_init(&q); // 큐 초기화
124     visited[v] = TRUE; // 정점 v 방문 표시
125     printf("%d 방문 -> ", v);
126     enqueue(&q, v); // 시작정점을 큐에 저장
127     while (!is_empty(&q)) {
128         v = dequeue(&q); // 큐에 저장된 정점 선택
129         for (w = g->adj_list[v]; w; w = w->link) //인접 정점 탐색
130             if (!visited[w->vertex]) { // 미방문 정점 탐색
131                 visited[w->vertex] = TRUE; // 방문 표시
132                 printf("%d 방문 -> ", w->vertex);
133                 enqueue(&q, w->vertex); //정점을 큐에 삽입
134             }
135     }
136 }
137
```



너비우선탐색의 시간 복잡도 분석

- 인접 행렬 $\rightarrow O(n^2)$
119번줄에서 n 번의 loop를 수행하므로 $O(n)$. n 개의 정점을 모두 방문해야 하므로 $n \cdot O(n) \rightarrow O(n^2)$
- 인접 리스트 $\rightarrow O(n+e)$
129번줄에서 각 정점에 연결되어 있는 간선의 개수만큼 탐색을 하게 되므로 시간복잡도 계산이 불가능 하다. 하지만 BFS가 다 끝난 후를 생각하면, 모든 정점을 한번씩 다 방문하고, 모든 간선을 한번씩 모두 검사했다고 할 수 있으므로 $O(n+e)$ 의 시간이 걸렸다고 할 수 있다.

```
108 void bfs_mat(GraphType* g, int v)
109 {
110     int w;
111     QueueType q;
112
113     queue_init(&q); // 큐 초기화
114     visited[v] = TRUE; // 정점 v 방문 표시
115     printf("%d 방문 -> ", v);
116     enqueue(&q, v); // 시작 정점을 큐에 저장
117     while (!is_empty(&q)) {
118         v = dequeue(&q); // 큐에 정점 추출
119         for (w = 0; w < g->n; w++) // 인접 정점
120             if (g->adj_mat[v][w] && !visited[w])
121                 visited[w] = TRUE; // 방문 표시
122                 printf("%d 방문 -> ", w);
123                 enqueue(&q, w); // 방문한 정점
124     }
125 }
126
```

```
118 void bfs_list(GraphType* g, int v)
119 {
120     GraphNode* w;
121     QueueType q;
122
123     queue_init(&q); // 큐 초기화
124     visited[v] = TRUE; // 정점 v 방문 표시
125     printf("%d 방문 -> ", v);
126     enqueue(&q, v); // 시작정점을 큐에 저장
127     while (!is_empty(&q)) {
128         v = dequeue(&q); // 큐에 저장된 정점 선택
129         for (w = g->adj_list[v]; w; w = w->link) //인접 정점 탐색
130             if (!visited[w->vertex]) { // 미방문 정점 탐색
131                 visited[w->vertex] = TRUE; // 방문 표시
132                 printf("%d 방문 -> ", w->vertex);
133                 enqueue(&q, w->vertex); //정점을 큐에 삽입
134             }
135     }
136 }
137
```



이제 나는 할 수 있다

1. 그래프란?
2. 그래프의 정의와 용어
3. 그래프의 표현(구현) 방법 (인접행렬, 인접리스트)
4. 그래프의 탐색
5. 깊이 우선 탐색
6. 너비 우선 탐색
7. 나는 깊이우선탐색에 대해 그림으로 그 과정을 그릴 수 있고 이 과정을 C코드로 작성할 수 있다
8. 나는 너비우선탐색에 대해 그림으로 그 과정을 그릴 수 있고 이 과정을 C코드로 작성할 수 있다
9. 나는 깊이우선탐색과 너비우선탐색에 대해 VS에서 한줄씩 수행하면서 변수창에서 자료구조가 그림으로 된 과정과 일치하는 지 확인할 수 있다

* 정답이 있는 연습문제 홀수문제 꼭 해결해보세요

